CLAUDE.md — Stealth Project Context

> This file is the single source of truth for any AI assistant (Claude Code, Cursor, etc.) working on this project. Read it fully before suggesting code, architecture, or features.

---

1. Project Identity

Name: Stealth **Tagline:** Private payroll with built-in compliance for DAO auditors. **One-liner:** Where DAO treasuries stay private, and audits stay possible.

Built for: Umbra Side Track — Solana Frontier Hackathon 2026 **Submission deadline:** ~26 May 2026 **Prize pool:** $10,000 USDC (1st: $5K, 2nd: $3K, 3rd: $2K)

---

2. The Problem We Solve

DAOs on Solana face an unsolved trilemma:

Current Option A: Realms / Squads	Current Option B: Tornado-style mixers
Audit-friendly	Not auditable
Every contributor's salary public forever	Total privacy
Burn rate, runway, vendor invoices all leak	Regulatory risk (sanctions, AML)
Talent retention problem	Cannot accept institutional capital

No serious DAO wants 100% public OR 100% anonymous. They need: **private by default, auditable on demand.**

That is exactly Umbra's value proposition — and Stealth is the application layer that makes it usable for real DAOs.

---

3. Our Solution: 3-Sided Platform

3.1 DAO Treasurer (the payer)

Pays contributors privately — without exposing individual salaries, vendor invoices, or internal patterns to competitors and the public.

3.2 Contributor (the payee)

Receives USDC into an encrypted balance. Their financial life is no longer public on Solscan.

3.3 Auditor (third-party firm)

Gets a role-based dashboard granted by the DAO. Generates industry-standard audit reports **without** the DAO needing to expose all data publicly.

This three-sided design is what makes Stealth defensible — anyone can build "private payroll." The auditor compliance layer is what nobody else has.

---

4. Why This Wins the Hackathon

1. **Uses every primitive Umbra offers** (Encrypted Balances, Mixer Pool, X25519 Compliance Grants, Mixer Pool Viewing Keys) — perfect SDK showcase.
2. **Cannot be built without Umbra** — competitors like Request Finance, Toku, Streamflow have public transactions, so they cannot offer this.
3. **Aligns 100% with Umbra's positioning** — "compliance-ready privacy," not Tornado-style absolute privacy.
4. **Clear revenue path for Umbra** — B2B DAO subscriptions + per-transaction SDK fees = recurring volume.
5. **Demo-able with a real pilot DAO** — far more credible than mockups.

---

5. Core User Flows

5.1 DAO Treasurer Flow

1. Set up DAO workspace, connect Squads multisig (read-only initially)
2. Invite teammates and add auditor firms
3. Pay contributors:
    - Upload CSV or pick from address book
    - Approve via multisig
    - Bulk private send via Umbra SDK
    - Tag transactions (engineering, marketing, ops, etc.)
4. Manage auditor access — grant scope (aggregate / per-category / full), set time bounds, revoke anytime

5.2 Contributor Flow (Minimal — by design)

1. Connect wallet
2. View encrypted earnings from all DAOs paying them
3. Withdraw to their own wallet anytime

That is it. Contributor side stays minimal. Product center of gravity is the auditor side.

5.3 Auditor Flow (The differentiator)

1. Login with auditor wallet
2. See list of DAOs that granted access
3. Open a DAO's audit dashboard:
    - Aggregate metrics (total disbursement, recipient count, category breakdown)
    - Sanctions screening status
    - Per-category drilldown (if authorized)
    - Per-transaction detail (if authorized)
    - Compliance flags (outliers, unusual patterns, OFAC results)
4. Export audit report (PDF, CSV, optional on-chain attestation in v2)

---

6. MVP Scope — STRICT Boundaries

What IS in scope (3-week MVP)

Week 1 — Foundation

- Next.js 14 + TypeScript + Tailwind setup
- Umbra SDK installation and client setup
- Wallet adapter (Phantom, Solflare via Wallet Standard)
- Basic deposit / withdraw flow working on devnet
- DAO workspace creation (mock data acceptable)
- Add team members + auditors (mock data acceptable)

Week 2 — Core Features

- Bulk private payment (CSV upload, parsing, validation)
- Transaction tagging system (category, memo)
- Auditor compliance grant generator (X25519)
- Auditor dashboard — aggregate view
- Real Umbra SDK integration end-to-end on devnet

Week 3 — Polish & Demo

- Per-category drilldown for auditor
- Audit report export (PDF + CSV)

- UI/UX polish
- Demo video + landing page
- Deploy to devnet (mainnet only if time permits)

What is OUT of scope (do NOT build these)

These are explicitly v2+. If the AI assistant suggests them, push back and stay focused.

- Contributor "proof of income" feature (originally pivoted away from)
- Bank / landlord / consumer verifier flow
- On-chain attestation by auditor
- Mobile native app
- Multi-DAO unified contributor view
- Recurring payments / scheduled disbursement
- Tax calculation per jurisdiction
- On/off-ramp integration
- Local currency display
- Subscription tiers
- Cross-chain support
- Decentralized identity (DID) integration

---

7. Tech Stack — Non-Negotiable

Layer	Tool	Notes
Frontend	Next.js 14 (App Router)	Server components for CSV processing, client components for wallet
Language	TypeScript	Strict mode on
Styling	Tailwind CSS	No CSS-in-JS, no styled-components
Wallet	Wallet Standard	Phantom, Solflare. NOT @solana/web3.js wallet adapter (deprecated)
Solana SDK	@solana/kit	Modern Solana TS library
Privacy	@umbra-privacy/sdk	The whole reason this product exists
CSV parsing	Papaparse	Browser-side parsing

Layer	Tool	Notes
PDF generation	react-pdf or jsPDF	For audit report export only
Deployment	Vercel	
Network	Solana devnet	Mainnet only after MVP works on devnet

Do NOT add

- State management libraries (Redux, Zustand) unless absolutely needed — start with React state
- ORM / database (the only "database" is on-chain Umbra state + encrypted local storage)
- Authentication libraries — wallet signature IS the auth
- Backend frameworks (Express, NestJS) — Next.js API routes are sufficient
- UI component libraries beyond shadcn/ui if used at all
- Animation libraries — Tailwind transitions are enough

---

8. Umbra SDK — How We Use It

8.1 Program IDs

- Mainnet: `UMBRAD2ishebJTcgCLkTkNUx1v3GyoAgpTRPeWoLykh`
- Devnet: `DSuKkyqGVGgo4QtPABfxKJKygUDACbUhirnuv63mEpAJ`

The SDK resolves automatically based on the `network` parameter.

8.2 Primitives We Use

Umbra Primitive	Used For
Encrypted Token Accounts	Contributor balances
Mixer Pool (UTXOs)	Unlinkable transfers from DAO to contributors
X25519 Compliance Grants	Role-based auditor access
Mixer Pool Viewing Keys	Auditor query capability

8.3 Critical SDK Concepts the AI Must Understand

- **Master seed derivation:** comes from wallet `signMessage` of `UMBRA_MESSAGE_TO_SIGN`.

Must NEVER modify the message — would invalidate all existing keys.

- **Registration is required** before any deposit/transfer can happen. New users must register first.
- **Encrypted balance != on-chain balance.** Reading balance requires decryption with master seed.
- **MPC operations have latency** because Arcium computes off-chain. UI must show loading states and handle retries.
- **Wallet must support both `solana:signTransaction` and `solana:signMessage` features** — error immediately if either is missing.

8.4 Reference Documentation

- Quickstart: https://sdk.umbraprivacy.com/quickstart
- Wallet adapters: https://sdk.umbraprivacy.com/sdk/wallet-adapters
- Registration: https://sdk.umbraprivacy.com/sdk/registration
- Deposit: https://sdk.umbraprivacy.com/sdk/deposit
- Transfers: https://sdk.umbraprivacy.com/sdk/transfers
- Compliance: https://sdk.umbraprivacy.com/sdk/compliance
- Mixer: https://sdk.umbraprivacy.com/sdk/mixer/overview

When in doubt about SDK usage, the AI should fetch and read these docs rather than guess.

---

9. Architecture Overview

```
┌─────────────────────────────────────────┐
│  Stealth Frontend (Next.js)             │
│  ├── /treasurer   → DAO admin dashboard │
│  ├── /contributor → Earnings view       │
│  └── /auditor     → Audit dashboard     │
└─────────────────────┬─────────────────────┘
                      │
                      ↓

┌─────────────────────────────────────────┐
│  @umbra-privacy/sdk                     │
│  ├── Solana RPC (transactions)          │
│  ├── Arcium MPC (encrypted compute)     │
│  └── Umbra Indexer (UTXO + Merkle proofs) │
└─────────────────────────────────────────┘
```

No backend database. State lives on-chain (via Umbra) plus minimal client-side cache for UX. If persistent storage is needed for non-sensitive metadata (e.g. DAO display name, auditor email for

invites), use a lightweight option — Vercel KV or Supabase free tier.

---

10. Code Style and Conventions

10.1 General principles

- **Boring code wins.** No clever abstractions. Hackathon judges and teammates need to read and trust the code in minutes.
- **Vertical slices, not layers.** Build one feature end-to-end before starting the next. Do not stub out 10 components for later.
- **TypeScript strict mode.** No `any` unless absolutely necessary, and document why.
- **One file per component.** Co-locate styles and logic.

10.2 File structure

```
/app
  /treasurer        — DAO admin pages
  /contributor      — Contributor pages
  /auditor          — Auditor pages
  /api              — Next.js API routes (only if needed)
/components
  /ui               — Reusable primitives
  /treasurer        — Treasurer-specific components
  /contributor
  /auditor
/lib
  /umbra            — Umbra SDK wrappers
  /utils            — Pure functions
/types              — Shared TypeScript types
```

10.3 Naming

- React components: `PascalCase` (`AuditorDashboard.tsx`)
- Hooks: `useCamelCase` (`useUmbraClient.ts`)
- Utility functions: `camelCase` (`parseCsvPayments.ts`)
- Types/Interfaces: `PascalCase` (`PaymentBatch`, `AuditScope`)
- Constants: `SCREAMING_SNAKE_CASE`

10.4 What to avoid

- `useEffect` for data fetching — use SWR or React Query if needed, but ideally server components

- Prop drilling more than 2 levels — lift state or use context
- Premature optimization — `useMemo`/`useCallback` only after measuring
- Inline secrets — all keys in `.env.local`, never committed
- `console.log` in production code — use a logger or strip them

---

11. Demo Script (60-second pitch)

Keep this in mind when building. Every feature should serve this story:

> "DAOs on Solana in 2026 have a strange problem. They want to be crypto-native, but their treasuries are public — competitors see burn rate, contributors see each other's salaries. The solution isn't Tornado Cash — that makes them un-auditable.
> **Stealth: private payroll for DAOs, with a compliance dashboard for their auditors.**
> [Demo: Treasurer view] Mira, treasurer of DAO X, pays 23 contributors at once. Private. One click.
> [Demo: Auditor dashboard] Hacken Auditing Firm has access. They log in — see the Q1 aggregate report. Total disbursement, category breakdown, sanctions screening. No individual salaries leaked.
> [Demo: Drill-down] Auditor needs engineering category detail? DAO grants scoped access — auditor sees 12 engineering transactions, recipients still encrypted. Enough for an audit, not enough to doxx.
> Built on Umbra. Private by default. Auditable on demand. The only payroll DAOs will need."

---

12. Critical "Don't Forget" Reminders for AI Assistants

When generating code or suggestions, the AI should:

1. **Always check Umbra SDK docs before guessing API shape.** The SDK is new (April 2026) and not in training data. Use web search or read documentation actively.
2. **Never invent SDK methods.** If unsure, ask or document the assumption clearly.
3. **Treat the auditor flow as the most important UX.** Most teams will build the treasurer side and stop there. The auditor dashboard is what wins.
4. **Stay on devnet until everything works.** Mainnet is not the goal of MVP — a working devnet demo is.
5. **Push back on scope creep.** If asked to add features in Section 6's "OUT of scope" list, remind the user this is v2.
6. **Demos > features.** Every code change should make the demo better. If it doesn't show in the demo, deprioritize it.
7. **Wallet UX matters.** Users will not retry if a wallet popup fails. Handle errors gracefully and show clear states.

8. **Privacy is the product.** Never log sensitive data, never embed viewing keys in URLs, never persist master seeds.

---

13. Pilot Customer Strategy

By demo day, ideally have **at least one real DAO** as a pilot user. Reach out early to:

- Superteam Indonesia
- Solana ID
- Small Solana grant programs
- Other hackathon teams who are themselves DAOs

A real pilot is worth more than any feature.

---

14. Who to Contact

- Hackathon coordinator: Telegram @abbasshaikh01
- Umbra team: via X @UmbraPrivacy or GitHub umbra-defi
- Hackathon listing: https://superteam.fun/earn/listing/umbra-side-track

---

15. Decision Log (update as we go)

Decision	Rationale	Date
Pivoted from "consumer income proof" to "DAO auditor compliance"	Auditors are crypto-native (no education burden), uses more Umbra primitives, clearer demo for Solana-focused judges	(initial planning)
Dropped contributor proof-of-income feature from MVP	Scope discipline — every additional feature dilutes execution quality	(initial planning)
Devnet-first strategy	Mainnet adds risk without demo benefit	(initial planning)

Add new decisions here as they happen — future-you and the AI will thank you.

End of CLAUDE.md. Read this file fully before any non-trivial code change.